

Folding-Direction HMM Drafter Notes

Speculative Decoding for ProstT5

June 9, 2026

1 High-Level Idea

The notebook `prostT5_folding_hmm.ipynb` studies the folding direction:

AA sequence -> 3Di structural-token sequence

This is different from the inverse-folding HMM setting. A standard protein Profile HMM is built from an amino-acid MSA and naturally emits amino-acid distributions:

Profile-HMM state -> $P(\text{AA})$

For folding, the drafter needs 3Di tokens:

AA input -> draft 3Di output

The MSA gives homologous AA sequences, not aligned 3Di labels. Therefore the folding-direction “HMM” in this notebook is not a strict HMMER Profile HMM. It is a family-local pseudo-3Di drafter.

2 Why HMM Cannot Directly Do Folding

In inverse folding, the task is:

3Di input -> AA output

This matches the emission type of a normal protein Profile HMM:

HMM match state -> amino-acid distribution

In folding, the task is:

AA input -> 3Di output

But a standard Profile HMM does not give:

HMM match state -> 3Di distribution

The workaround is to generate pseudo-3Di labels for family homologs using ProstT5, then learn a family-conditioned 3Di drafter from those pseudo-labels.

3 Notebook Flow

3.1 Setup And Configuration

The notebook first imports packages, checks the CUDA runtime, and defines experiment parameters.

Important parameters:

- K_VALUES: draft lengths tested by assisted generation.
- BENCHMARK_PROTEIN_LIMIT: number of proteins in the benchmark.
- RUN_BENCHMARKS: whether to run Part 2 timing experiments.
- FAMILY_GENERALIZATION_TRAIN_SEQS: training homolog count in Part 1.
- FAMILY_GENERALIZATION_HOLDOUT_PER_FAMILY: held-out homolog count in Part 1.
- FAMILY_MSA_MAX_SEQS: maximum homolog rows used for the per-protein family drafter.

Key code:

```
REQUIRE_CUDA_RUNTIME = True
if REQUIRE_CUDA_RUNTIME and device.type != "cuda":
    raise RuntimeError("This notebook is configured for Colab GPU/CUDA.")

K_VALUES = [1, 3, 5, 8]
BENCHMARK_PROTEIN_LIMIT = 10
FAMILY_GENERALIZATION_TRAIN_SEQS = 32
FAMILY_GENERALIZATION_HOLDOUT_PER_FAMILY = 2
```

3.2 Tokenization Helpers

The folding prompt is:

<AA2fold>

The assistant drafter predicts over the 20 3Di symbols, but HuggingFace generation expects logits over the full ProstT5 vocabulary. The bridge is:

```
THREEDI_TOKEN_IDS = [
    tokenizer.encode(f" {tok}", add_special_tokens=False)[0]
    for tok in THREEDI_ALPHABET
]
```

This maps:

3Di symbol -> ProstT5 vocabulary ID

4 Part 1: Mini Family Test

Part 1 asks whether the family pseudo-HMM learns family-specific signal. It is not a speed benchmark and does not use HuggingFace assisted generation.

4.1 Drafter Used In Part 1

Part 1 uses:

FamilyAA3DiHMM

This is a lightweight length-general HMM-like model over pseudo-3Di states. It learns:

- start probabilities,
- transition probabilities between 3Di states,
- AA emissions from pseudo-3Di states.

It is used only for the in-family versus out-family sanity check.

4.2 Family Selection

The two seed families are:

```
FAMILY_GENERALIZATION_SEED_IDS = ["P01542", "POA7N4"]
```

Each family contributes:

- 32 training homologs by default,
- 2 held-out homologs by default.

4.3 Candidate Homolog Selection

The code skips the query row and exact duplicate sequences:

```
records = _parse_a3m_records(a3m) if a3m else []
rows = []
seen = {aa_seq.upper()}

for name, raw_aln in records[1:]:
    aa = _ungap_aa(_strip_a3m_insertions(raw_aln))
    if len(aa) < FAMILY_MSA_MIN_SEQ_LEN or aa in seen:
        continue
    seen.add(aa)
    rows.append({"member_id": name, "aa": aa})
```

This avoids using the seed/query sequence itself as a held-out or training homolog.

4.4 Train And Held-Out Split

The selected examples are split as:

```
needed = train_n + holdout_n
selected = candidates[:needed]
examples = _load_or_fold_family_examples(seed_uid, selected)

holdouts = examples[:holdout_n]
train = examples[holdout_n:holdout_n + train_n]
```

With the default settings:

```
examples[0:2] -> held-out test homologs
examples[2:34] -> training homologs
```

4.5 Pseudo-3Di Labeling

If a pseudo-label is not cached, the notebook folds the homolog with ProstT5:

```
if aa not in by_aa:
    di = generate_folding_reference(aa)
    by_aa[aa] = {
        "member_id": row["member_id"],
        "aa": aa,
        "3di": di,
    }
```

Conceptually:

homolog AA -> ProstT5 -> pseudo-3Di

4.6 Part 1 Metrics

Each held-out homolog is evaluated against both family models:

```
relation = "in_family" if test_family == model_family else "out_family"
```

The metrics are:

- `heldout_index`: local index of the held-out homolog.
- `heldout_member_id`: MSA header/name of the held-out homolog.
- `token_acc`: fraction of positions where argmax 3Di matches the pseudo-label.
- `mean_ref_logp`: average log probability assigned to the pseudo-label 3Di token.

Key scoring code:

```
pred_idx = logits[:,L].argmax(axis=1)
token_acc = float((pred_idx[valid] == ref_idx[valid]).mean())

mean_ref_logp = float(
    logits[np.arange(L)[valid], ref_idx[valid]].mean()
)
```

Expected meaningful result:

```
in_family score > out_family score
```

5 Part 2: Assisted-Generation Benchmark

Part 2 asks whether the folding-direction drafter speeds up ProstT5 generation while preserving the plain ProstT5 greedy output.

5.1 Static FamilyFoldingHMMDrafter

The static drafter is:

FamilyFoldingHMMDrafter

It builds one family-specific position table per query protein:

position i -> P(3Di token)

At generation time it returns a slice:

```
def get_draft_logits(self, pos: int, prefix=None, K: int | None = None):
    if K is None:
        return self.logits[pos:]
    return self.logits[pos:pos + K]
```

For example:

```
pos = 10, K = 5
return positions 10, 11, 12, 13, 14
```

It does not look back at previously generated tokens.

5.2 Prefix-Aware Folding HMM

The prefix-aware drafter adds local 3Di context:

family + position + last p generated 3Di tokens -> P(next 3Di)

The new parameter is:

- p: number of previous generated 3Di tokens used as prefix context.

Example:

```
p = 3
prefix = [..., a, d, y]
context = (a, d, y)
```

If the context is missing in the count table, it falls back to the static distribution:

```
return self.logits[pos]
```

5.3 Assistant Model Wrappers

The static and prefix-aware drafters are wrapped as HuggingFace assistant models:

```
model.generate(..., assistant_model=hmm_assistant)
```

The wrapper maps the 20 3Di logits into the full ProstT5 vocabulary:

```
logits[0, pos, THREEDI_TOKEN_IDS] = row
```

5.4 Part 2 Metrics

The benchmark records:

- wall_s: runtime in seconds.
- speedup: plain ProstT5 runtime divided by assisted runtime.
- peak_vram_gb: peak GPU memory.
- exact_match: whether assisted output equals plain ProstT5 greedy output.
- family_projected: number of usable projected homolog pseudo-label rows.

6 Part 1 Versus Part 2

Part	Question	Main model
Part 1	Is the family drafter meaningful?	FamilyAA3DiHMM
Part 2	Does it speed up ProstT5 correctly?	Assistant HMM drafters

Short version:

Part 1 = drafter sanity check

Part 2 = actual assisted-generation benchmark

7 Current Run Result Explanation

The current run produced result lines such as:

```
prefix_exact=8/20, median_speedup=1.16x, family_projected=16
```

This line is from the prefix-aware Part 2 benchmark for one protein. It summarizes all prefix-aware settings tested for that protein.

7.1 What The Numbers Mean

- **family_projected=16**: the drafter successfully used 16 homolog pseudo-3Di rows from the query protein's MSA family. This is a construction-quality signal. If this number is low, the family drafter has little data.
- **median_speedup=1.16x**: across the prefix-aware configurations for this protein, the median assisted-generation runtime was 1.16 times faster than plain ProstT5 greedy folding. This is a modest positive speedup.
- **prefix_exact=8/20**: only 8 out of 20 prefix-aware assisted-generation configurations produced exactly the same 3Di sequence as plain ProstT5 greedy folding.

The 20 configurations come from:

```
P_VALUES = [1, 2, 3, 4, 5]
```

```
K_VALUES = [1, 3, 5, 8]
```

```
5 * 4 = 20
```

7.2 What Is Positive

The positive part is that the folding-direction pseudo-HMM can be built from family data and can sometimes speed up generation. The **family_projected=16** value means the MSA-to-pseudo-3Di projection pipeline worked for 16 homolog rows. The **median_speedup=1.16x** value means the assistant path was faster than the plain verifier for the median tested setting.

This supports the claim that the family-local pseudo-3Di drafter is meaningful enough to test as a folding-direction drafter.

7.3 What Is Concerning

The concerning part is **prefix_exact=8/20**. In strict greedy speculative decoding, the drafter is allowed to propose wrong tokens, but the verifier should reject wrong tokens and force the final sequence to match the plain ProstT5 greedy output.

Therefore:

Drafter can be wrong: acceptable.

Final assisted output differs from verifier: correctness issue.

Only 8 exact configurations out of 20 means that the current folding assistant setup is not yet correctness-clean. The likely causes are assistant-logit alignment, token-position alignment, HuggingFace assistant-model behavior, or generation configuration details.

7.4 How To Report The Current Result

A careful report should say:

The current folding-direction pseudo-HMM drafter can be constructed from MSA-family pseudo-labels and shows modest speedup in some settings. For example, one prefix-aware run reports `family_projected=16` and `median_speedup=1.16x`. However, exact agreement with plain ProstT5 greedy generation was only 8/20 across prefix length and draft length settings, so the current assisted-generation path should be treated as preliminary rather than a fully correct speculative-decoding implementation.

7.5 Interpretation For The Slides

The slide-level conclusion is:

- The drafter construction works: family homologs can be pseudo-labeled and projected.
- The speedup signal exists but is small so far.
- The exactness issue is the main blocker.
- The result is useful as a preliminary folding-direction drafter experiment, not yet as a final speculative-decoding result.

8 Outputs

Part 1 saves:

```
prostT5/results/folding_hmm_results/family_generalization/family_cross_test.csv
```

Part 2 saves:

```
prostT5/results/folding_hmm_results/folding_hmm_results.csv
prostT5/results/folding_hmm_results/folding_prefix_hmm_results.csv
prostT5/results/folding_hmm_results/folding_hmm_plots.png
```